LAB SHEET NO. 9

OBJECT DETECTION AND RECOGNITION USING DEEP LEARNING MODELS ON STANDARD IMAGE DATASETS

Name :- Jai Pratap Rajput

CU ID : CU24250071

BTech CSE 3rd Year SECTION - "C"

Roll No. :- 27

AIM

To implement a deep learning-based object detection and recognition system using a pre-trained model and evaluate its performance for identifying and localizing multiple objects in an image.

GITHUB LINK:-https://github.com/DevSharma1912/Computer_vision/tree/main/ComputerVision/Experiment9

STEP 1: IMPORT REQUIRED LIBRARIES

```
!pip install ultralytics
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO
import tkinter as tk
from tkinter import filedialog
```

```
Collecting ultralytics
  Downloading ultralytics-8.4.157-py3-none-any.whl.metadata (46 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.7/46.7 kB 1.7 MB/s eta 0:00:00
Requirement already satisfied: cloudpickle>=3.1.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.1.2)
Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.32.5)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python!=4.13.0.90,>=4.7.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.0.0.93)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.11.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.26.0+cpu)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (1.35.2)
Collecting nvidia-ml-py>=12.0.0 (from ultralytics)
  Downloading nvidia_ml_py-13.610.43-py3-none-any.whl.metadata (9.7 kB)
Collecting ultralytics-thop>=2.1.6 (from ultralytics)
  Downloading ultralytics_thop-2.1.6-py3-none-any.whl.metadata (13 kB)
Collecting ultralytics-platform>=0.1.45 (from ultralytics)
  Downloading ultralytics_platform-0.1.52-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.64.0
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.13/dist-packages (from polars>=0.20.0->ultralytics) (1
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2026.7
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (4.1
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (2025.12.0)
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages (from ultralytics-platform>=0.1.45->ultralytics)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultra
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.4
```

```
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx<1,>=0.28->ultralytics-plat
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultraly
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralyti
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->torch>=1.8.0->ultralytics) (3.0.3
Downloading ultralytics-8.4.157-py3-none-any.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 20.2 MB/s eta 0:00:00
Downloading nvidia_ml_py-13.610.43-py3-none-any.whl (53 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.2/53.2 kB 2.1 MB/s eta 0:00:00
Downloading ultralytics_platform-0.1.52-py3-none-any.whl (88 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.0/88.0 kB 4.7 MB/s eta 0:00:00
Downloading ultralytics_thop-2.1.6-py3-none-any.whl (30 kB)
Installing collected packages: nvidia-ml-py, ultralytics-thop, ultralytics-platform, ultralytics
Successfully installed nvidia-ml-py-13.610.43 ultralytics-8.4.157 ultralytics-platform-0.1.52 ultralytics-thop-2.1.6
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
```

STEP 2: LOAD PRE-TRAINED YOLO MODEL

```
# Load the pre-trained YOLOv8 model.
# YOLOv8 is trained on the COCO dataset.

model = YOLO("yolov8n.pt")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 71.9MB/s 0.
```

STEP 3: SELECT IMAGE FROM COMPUTER

```
from google.colab import files
import io
import numpy as np

print("Please upload an image file.")
uploaded = files.upload()

if not uploaded:
    raise ValueError("No image selected.")

# Assuming only one file is uploaded for simplicity
filename = next(iter(uploaded))
image_bytes = uploaded[filename]

# Read the image bytes using OpenCV
image = cv2.imdecode(np.frombuffer(image_bytes, np.uint8), cv2.IMREAD_COLOR)

if image is None:
    raise ValueError("Unable to read the selected image.")

image_rgb = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)
```

```
Please upload an image file.
Choose Files Screenshot... 223749.png
Screenshot 2025-05-01 223749.png(image/png) - 498136 bytes, last modified: 5/1/2025 - 100% done
Saving Screenshot 2025-05-01 223749.png to Screenshot 2025-05-01 223749.png
```

STEP 4: DISPLAY ORIGINAL IMAGE

```
plt.figure(figsize=(8, 6))

plt.imshow(image_rgb)

plt.title("Original Image")

plt.axis("off")

plt.show()
```



STEP 5: IMAGE PREPROCESSING

```
resized_image = cv2.resize(
    image_rgb,
    (640, 640)
)
```

STEP 6: DISPLAY PREPROCESSED IMAGE

```
plt.figure(figsize=(8, 6))

plt.imshow(resized_image)

plt.title("Preprocessed Image")

plt.axis("off")

plt.show()
```



STEP 7: PERFORM OBJECT DETECTION

```
results = model(image_rgb)

result = results[0]
```

```
0: 384x640 3 persons, 1 baseball bat, 2 baseball gloves, 549.5ms
```

```
Speed: 25.4ms preprocess, 549.5ms inference, 52.8ms postprocess per image at shape (1, 3, 384, 640)
```

STEP 8: EXTRACT DETECTED OBJECTS

```
boxes = result.boxes

detected_objects = []

for box in boxes:

    class_id = int(box.cls[0])

    confidence = float(box.conf[0])

    class_name = model.names[class_id]

    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0]
    )

    detected_objects.append(
        [
            class_name,
            confidence,
            x1,
            y1,
            x2,
            y2
        ]
    )
```

STEP 9: DISPLAY DETECTED OBJECTS

```
annotated_image = result.plot()

annotated_image = cv2.cvtColor(
    annotated_image,
    cv2.COLOR_BGR2RGB
)

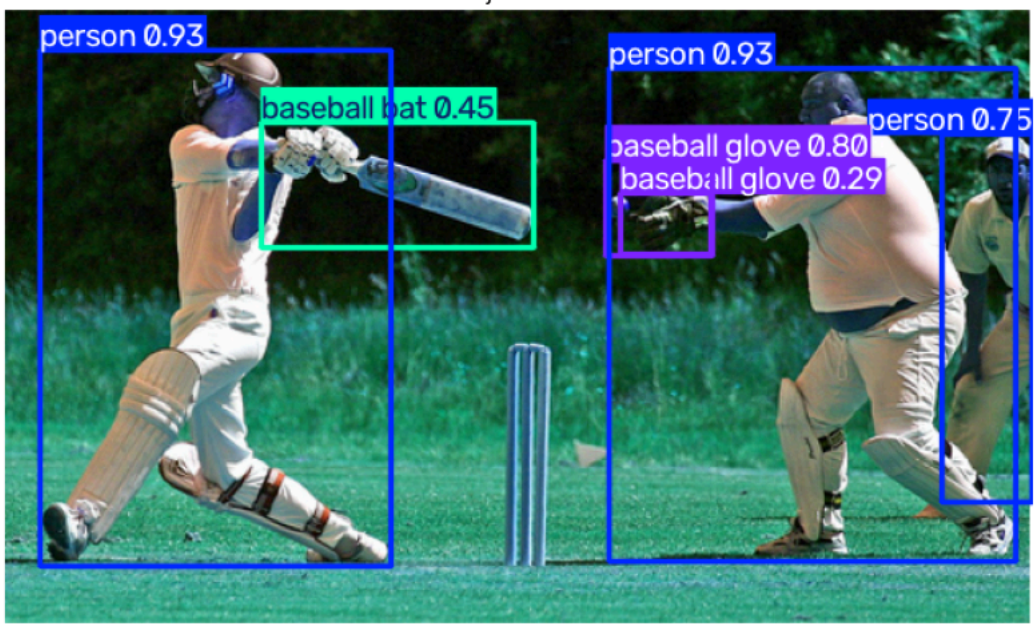
plt.figure(figsize=(10, 8))

plt.imshow(annotated_image)

plt.title(
    "YOLO Object Detection"
)

plt.axis("off")

plt.show()
```



STEP 10: COUNT DETECTED OBJECTS

```
object_counts = {}

for obj in detected_objects:

    class_name = obj[0]

    if class_name in object_counts:

        object_counts[class_name] += 1

    else:

        object_counts[class_name] = 1
```

STEP 11: DISPLAY OBJECT COUNT

```
plt.figure(figsize=(10, 5))

if len(object_counts) > 0:

    plt.bar(
        object_counts.keys(),
        object_counts.values()
    )

    plt.xlabel("Object Class")

    plt.ylabel("Number of Objects")

    plt.title(
        "Detected Object Count"
    )

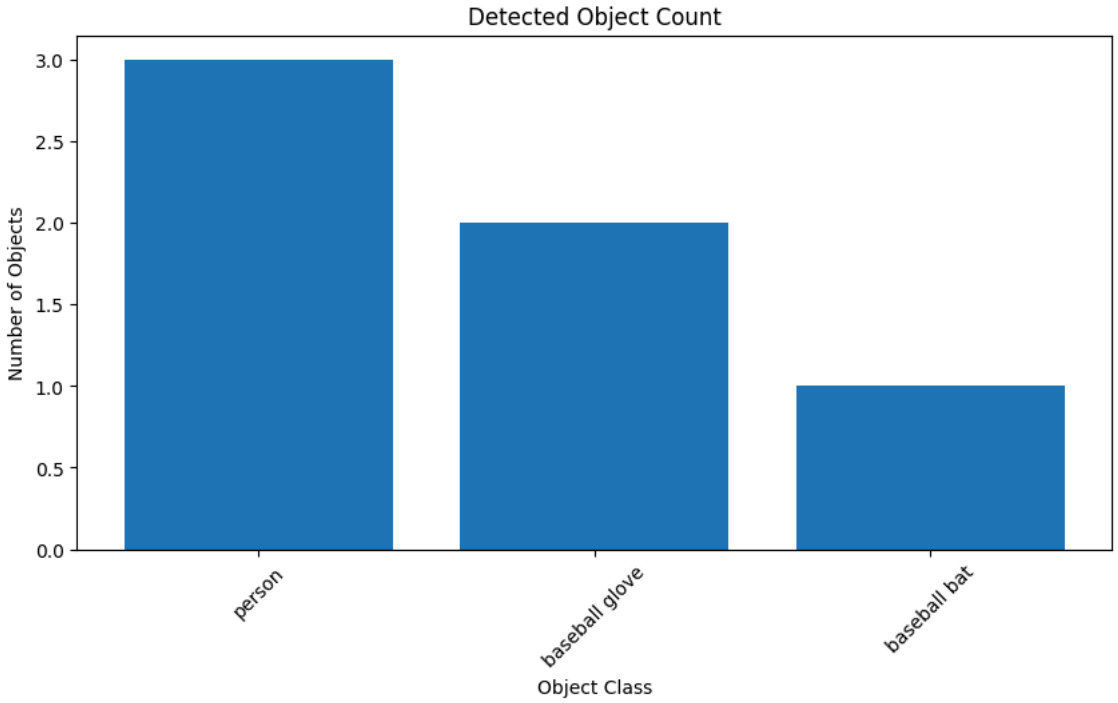
    plt.xticks(rotation=45)

else:

    plt.text(
        0.5,
        0.5,
        "No Objects Detected",
        ha="center",
        va="center"
    )
```

```
    plt.axis("off")

plt.show()
```



STEP 12: CALCULATE DETECTION STATISTICS

```
total_objects = len(detected_objects)

if total_objects > 0:

    average_confidence = np.mean(
        [
            obj[1]
            for obj in detected_objects
        ]
    )

else:

    average_confidence = 0

inference_time = result.speed["inference"]
```

STEP 13: DISPLAY DETECTION SUMMARY

```
summary = [
    ["Total Objects Detected", total_objects],
    ["Average Confidence", round(average_confidence, 3)],
    ["Inference Time (ms)", round(inference_time, 2)]
]

fig, ax = plt.subplots(figsize=(7, 2.5))

ax.axis("off")

table = ax.table(
    cellText=summary,
    colLabels=["Parameter", "Value"],
    loc="center"
)

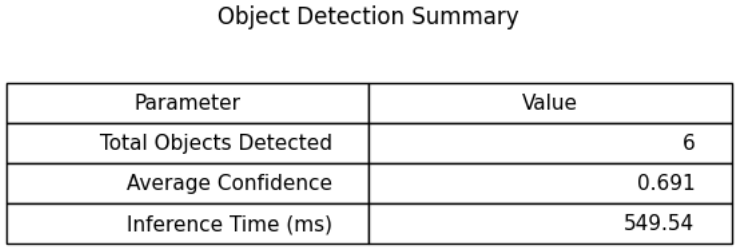
table.auto_set_font_size(False)
```

```
table.set_fontsize(11)

table.scale(1, 1.8)

plt.title(
    "Object Detection Summary"
)

plt.show()
```



Parameter	Value
Total Objects Detected	6
Average Confidence	0.691
Inference Time (ms)	549.54

QUESTIONS AND ANSWERS

Q1. Differentiate between image classification, object detection and image segmentation.

Answer: Image classification assigns a class label to the complete image.

Object detection identifies objects and provides their class labels and bounding boxes.

Image segmentation assigns labels to individual pixels or regions of an image.

Q2. Explain the working principle of YOLO.

Answer: YOLO stands for You Only Look Once.

It processes the complete image using a deep learning neural network and predicts bounding boxes, class labels and confidence scores.

YOLO performs detection in a single main detection process, which makes it suitable for fast object detection.

Q3. Compare YOLO, SSD and Faster R-CNN based on speed, accuracy and practical applications.

Answer: YOLO and SSD are single-stage object detection methods designed for fast detection.

Faster R-CNN is a two-stage object detection method that first generates region proposals and then performs classification and localization.

YOLO and SSD are commonly suitable for applications where fast inference is important, while Faster R-CNN uses a more computationally intensive two-stage approach.

Q4. What is a bounding box? Why is it important?

Answer: A bounding box is a rectangular box surrounding a detected object in an image.

It is important because it provides the location and approximate size of the detected object.

Q5. Explain the significance of confidence score and IoU.

Answer: A confidence score represents the model's confidence in a particular object detection.

IoU stands for Intersection over Union.

It measures the overlap between the predicted bounding box and the ground-truth bounding box.

A higher IoU indicates greater overlap between the predicted and actual bounding boxes.

Q6. What are the advantages of using pre-trained deep learning models over traditional image processing?

Answer: Pre-trained deep learning models have already learned useful visual features from large datasets.

They reduce the need to train a complete model from the beginning and can identify complex patterns.

Traditional image processing generally depends more on manually designed features and rules.

Q7. Why are datasets such as MS COCO and Pascal VOC widely used?

Answer: MS COCO and Pascal VOC contain labeled images with different object categories and localization information.

They are widely used for training, testing and evaluating object detection algorithms.

Q8. Mention five real-world applications of object detection.

Answer:

1. Autonomous vehicles
2. Surveillance systems
3. Healthcare

4. Retail analytics
5. Industrial inspection

Q9. What challenges are commonly encountered while detecting objects in complex real-world environments?

Answer:

Common challenges include:

1. Poor lighting
2. Object occlusion
3. Overlapping objects
4. Small objects
5. Complex backgrounds
6. Different object sizes
7. Camera movement
8. Low-quality images

Q10. How can object detection systems be further improved?

Answer:

Object detection systems can be improved using:

1. Larger and more diverse datasets
2. Data augmentation
3. Improved deep learning architectures
4. Better feature extraction
5. Improved training techniques
6. Model optimization
7. Better hardware
8. Recent computer vision techniques

RESULT

The object detection and recognition system was successfully implemented using a pre-trained YOLO deep learning model.